

The Remote Guard: Confidentiality Enforcement for Repository Publication in zzcollab

2026-06-28 10:16 PDT

Reference commit: dbebb09 on branch feat/init-setup-flow.

Abstract

This paper documents the design and implementation of the ‘remote guard’, a confidentiality-enforcement mechanism added to zzcollab to prevent the accidental publication of repositories that must remain local-only. zzcollab scaffolds research compendia and can, on request, create a remote repository on a git forge (GitHub or GitLab) and push to it. For a class of projects subject to data-governance or confidentiality restrictions, that capability is a liability: a single mistaken invocation can expose a private compendium on a public forge, and such exposure is effectively irreversible because the forge may cache or index the content before any deletion. We describe the threat the guard addresses, the three independent deny signals it evaluates, its fail-closed evaluation order, the two enforcement points at which it is consulted, and the verification performed. We are explicit about what was validated by direct execution and what was confirmed only by source inspection.

1. Motivation

1.1 The exposure problem

zzcollab’s remote-creation commands, `cmd_github` and `cmd_gitlab`, are convenience wrappers over the `gh` and `glab` forge CLIs. They initialise git if necessary, create a remote repository, and push. The operation is outward-facing and hard to reverse: once a commit reaches a public or institution-visible forge, it may be replicated, cached, or indexed before the author recognises the error. Deleting the remote afterward does not guarantee the content is gone.

This is acceptable for the common case, where the author intends to publish. It is not acceptable for projects that carry a standing confidentiality constraint. In the maintainer’s environment, every repository beneath a designated subtree (`~/prj/srv/`) is subject to such a constraint and must never acquire a remote. The risk is not malice but muscle memory: the same `zgc github` command that is correct in one project directory is a confidentiality breach one directory over. A control that depends on the operator remembering the distinction at the moment of invocation is not a control.

1.2 Design objectives

The guard was designed against four objectives.

- Fail closed. Any single positive deny signal must block the operation, regardless of the others. Ambiguity resolves to denial, not permission.
- Defence in depth. No single mechanism should be the sole barrier. A marker file, a configuration flag, and a path-prefix rule each suffice independently, so that the loss or misconfiguration of one does not open the path.
- Locality of declaration. The constraint should be expressible where it is most robust: travelling with the project (a marker file), in user or project configuration, or as a machine-wide path policy that no per-project action can override.
- Symlink soundness. The path-based rule must compare real, resolved paths on both sides, because the protected subtree is commonly reached through a symlink (for example a cloud-storage mount), and a naive string comparison on the symlink path would silently fail to match.

2. Design

2.1 The three deny signals

The guard is implemented as a single function, `remote_allowed`, in `modules/config.sh`. It accepts an optional directory argument (defaulting to the current working directory) and returns 0 when remote creation is permitted and 1 when it is denied. On denial it sets a module -level variable, `ZZCOLLAB_REMOTE_DENY_REASON`, to a human-readable cause so the caller can tell the operator which signal fired.

Three independent signals are evaluated.

1. Marker file. A file named `.zzcollab-no-remote` in the project directory denies the remote. This signal travels with the project, requires no configuration, and is the most local declaration available. It is checked first, before any configuration is loaded, so a marked project is protected even if the configuration system is misconfigured or unavailable.
2. Configuration opt-out. The configuration key `remote.allow`, when set to the string `false` at either user or project level, denies the remote. This permits a blanket opt-out for an account or a project without touching the filesystem of every compendium.
3. Blocked-path prefix. The configuration key `remote.blocked_paths` holds a space-separated list of path prefixes. If the project's resolved path lies at or beneath any listed prefix, the remote is denied. This is the machine-wide policy lever: a single user-level entry naming the confidential subtree protects every present and future project beneath it, with no per-project action required. This is the signal that enforces the `~/prj/srv/` constraint.

2.2 Evaluation order and fail-closed semantics

The signals are evaluated in the order above, and the first positive deny signal returns immediately. The ordering is deliberate rather than incidental.

The marker-file check precedes configuration loading. This is the strongest guarantee the guard offers: a project bearing the marker is denied a remote even if `load_config` would fail, even if no configuration file exists, and even if the configuration would otherwise permit the remote. The check reduces to the presence of a single file, which is the most difficult precondition to subvert by accident.

The two configuration-backed signals are evaluated only after the marker check passes. They call `load_config`, which reads the user-level configuration first and the project-level configuration second, so that a project may tighten but the evaluation never depends on a single layer being present. A failure to load configuration is tolerated for these signals; only the marker check is wholly independent of the configuration subsystem.

The function returns 0 only after all three signals decline to fire. There is no path by which an unevaluated or errored signal yields permission: every early return is a denial, and the sole success return is the final statement of the function.

2.3 Symlink resolution in the path check

The blocked-path comparison resolves symlinks on both operands before comparing. The project directory is resolved with `cd "$dir" && pwd -P`, which yields the physical path with all symlink components expanded. Each configured prefix is expanded similarly when it names an existing directory, and a leading tilde in a configured prefix is expanded to the home directory first.

The comparison then tests whether the resolved project path equals a resolved prefix or begins with that prefix followed by a path separator. The separator guard is necessary to avoid a false match between, for example, `/home/user/srv-public` and a prefix of `/home/user/srv`.

Resolving both sides is what makes the rule sound in the maintainer's environment, where the working tree is reached through a cloud-storage symlink whose physical target lies under the protected subtree. A comparison on the unresolved symlink path would not match the configured physical prefix, and the guard would silently permit a remote it was configured to deny. Resolving to physical paths closes that gap.

3. Enforcement points

The guard is consulted at two points.

3.1 Command entry

`cmd_github` and `cmd_gitlab` both call `remote_allowed` as their first action, before any git or forge operation. When the guard denies, the command logs the operation as disabled, reports `ZZCOLLAB_REMOTE_DENY_REASON` as the cause, and returns a non-zero status without creating git history, contacting the forge, or pushing. This is the authoritative enforcement point: regardless of how the command was reached, including a direct `zcc github` on the command line, the guard runs first.

3.2 The initialisation wizard

The init-time feature wizard (`run_feature_wizard` in `modules/toggle.sh`) offers an opt-in ‘remote’ item only when remote creation can actually succeed. The item is shown only when a real forge is configured, the session is interactive, the relevant forge CLI is installed, and `remote_allowed` permits the operation. For a confidential project the guard denies, and the item is hidden from the wizard entirely; it cannot be toggled on. This is a usability refinement, not a security boundary: the wizard ultimately dispatches to `cmd_github` or `cmd_gitlab`, which enforce the guard regardless. Hiding the item spares the operator a choice that would be refused, but the refusal does not depend on the item being hidden.

4. Configuration surface

The guard introduces two configuration keys, both registered in the configuration map in `modules/config.sh` and therefore settable through the standard `zcc config set` interface and readable through `zcc config get`.

- `remote.allow` (`CONFIG_REMOTE_ALLOW`): accepts `true` or `false`. The setter validates the value and rejects anything else, so a typo cannot silently disable the opt-out.
- `remote.blocked_paths` (`CONFIG_REMOTE_BLOCKED_PATHS`): a space-separated list of path prefixes.

The marker file requires no configuration and has no key; its presence in the project directory is its entire interface.

5. Verification

The guard’s logic was exercised by direct execution against scratch projects, with the user-configuration layer isolated to a nonexistent file so that the host environment could not influence the outcome. Seven cases were run.

Case	Setup	Expected	Observed
clean project	no marker, no config	allow	allow
marker file	<code>.zzcollab-no-remote</code> present	deny	deny
<code>remote.allow</code> false	project config sets it false	deny	deny
<code>remote.allow</code> true	project config sets it true	allow	allow
blocked-path match	project under a configured prefix	deny	deny
blocked-path unrelated	prefix configured, project elsewhere	allow	allow
symlinked path resolved	project reached via symlink, real path under the configured prefix	deny	deny

All seven produced the expected result, and in each deny case the reported reason matched the signal under test. The symlinked-path case is the load-bearing one for the `~/prj/srv/` constraint: a project reached through a symlink whose physical target lay under the configured prefix was denied, confirming that the `pwd -P` resolution makes the rule sound under the cloud-storage mount it was designed for. The fail-closed ordering was confirmed incidentally: the marker-file case denied before any configuration was consulted.

5.1 Limits of the verification

The following were confirmed by source inspection only, not by execution.

- End-to-end refusal through `cmd_github` and `cmd_gitlab`. The guard function was exercised directly; the callers were read and confirmed to invoke it as their first action and to return on denial, but the full command paths were not driven to a live forge.
- Wizard suppression. The condition that hides the ‘remote’ item was read and confirmed to include `remote_allowed`, but the interactive wizard was not driven.
- User-layer precedence. The verification isolated the user layer to remove host influence. The interaction of a user-level `remote.allow` with a project-level override was reasoned about but not exercised.

6. Assessment and limitations

The guard is a sound defence against accidental publication, which is the threat it was built for. It is not, and is not intended to be, a defence against a determined operator: any of the three signals can be removed by the same person who can run the publication command, and the guard does not prevent a manually configured `git remote add` followed by a direct `git push`, which bypass `cmd_github` and `cmd_gitlab` entirely. The control raises the floor against mistakes; it does not constitute access control.

Two observations bear on future work. First, because the guard is enforced only at the `zzcollab` command layer, a project that must remain confidential is protected only against `zzcollab`-mediated publication; a complementary control at the `git` layer, such as a `pre-push` hook installed at scaffold time, would extend the guarantee to direct `git` use. Second, the blocked-path policy lives in user configuration, which a project-level configuration cannot override for tightening but which an operator can loosen by editing their own file; for environments where the policy must be immutable to the operator, a system-level configuration path would be the appropriate location.

Rendered on 2026-06-28 at 10:16 PDT. Source: ~/prj/sfw/07-zzcollab/zzcollab/docs/remote-guard-whitepaper.md